

Homework 2: [YOUR NAME HERE]

Introduction to Time Series, Fall 2024

2001-09-27

The total number of points possible for this homework is 39. The number of points for each question is written below, and questions marked as “bonus” are optional (points awarded for bonus problems can be used to earn back points that you may have lost on other parts of this homework but will not put you above full credit). Submit the **knitted pdf file** from this Rmd to Gradescope.

If you collaborated with anybody for this homework, put their names here:

Simple regression

1. (2 pts) Derive the population least squares coefficients, which solve

$$\min_{\beta_1, \beta_0} \mathbb{E}[(y - \beta_0 - \beta_1 x)^2],$$

by differentiating the criterion with respect to each β_j , setting equal to zero, and solving. Repeat the calculation but without intercept (without the β_0 coefficient in the model).

2. (2 pts) As in Q1, now derive the sample least squares coefficients, which solve

$$\min_{\beta_1, \beta_0} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2.$$

Again, repeat the calculation but without intercept (no β_0 in the model).

3. (2 pts) Prove or disprove: in the model without intercept, the regression coefficient of x on y is the inverse of that from the regression of y on x . Answer the question for both the population version and the sample version.

4. (3 pts) Consider the following hypothetical. Let y be the height of a child and x be the height of their parent, and consider a regression of y on x , performed in a large population. Suppose that we estimate the regression coefficients separately for male and female parents (two separate regressions) and we find that the slope coefficient from the former regression $\hat{\beta}_1^{\text{dad}}$ is smaller than that from the latter $\hat{\beta}_1^{\text{mom}}$. Suppose however that we find (in this same population) the sample correlation between a father's height and their child's height is *larger* than that between a mother's height and their child's height. What is a plausible explanation for what is happening here?

Multiple regression

5. (2 pts) In class, we claimed that the multiple regression coefficients, with respect to responses y_i and feature vectors $x_i \in \mathbb{R}^p$, $i = 1, \dots, n$, can be written in two ways: the first is

$$\hat{\beta} = \left(\sum_{i=1}^n x_i x_i^T \right)^{-1} \sum_{i=1}^n x_i y_i.$$

The second is

$$\hat{\beta} = (X^T X)^{-1} X^T y,$$

where $X \in \mathbb{R}^{n \times p}$ is a feature matrix, with i^{th} row x_i , and $y \in \mathbb{R}^n$ is a response vector, with i^{th} component y_i . Prove that these two expressions are equivalent.

6. (Bonus) Derive the population and sample multiple regression coefficients by solving the corresponding least squares problem (differentiating the criterion with respect to each β_j , setting equal to zero, and solving). For the sample least squares coefficient, deriving either representation in Q5 will be fine.

Covariance calculations

7. (3 pts) Let $x \in \mathbb{R}^n$ and $y \in \mathbb{R}^m$ be random vectors, and let $A \in \mathbb{R}^{k \times n}$ and $B \in \mathbb{R}^{\ell \times m}$ be fixed matrices. Prove that

$$\text{Cov}(Ax, By) = A \text{Cov}(x, y) B^T.$$

Prove as a consequence that $\text{Cov}(Ax) = A \text{Cov}(x) A^T$. Hint: you may use the rule for covariances of linear combinations (as reviewed in the lecture from week 2, "Measures of dependence and stationarity").

Proof.

By definition of the cross-covariance matrix:

$$\text{Cov}(Ax, By) = E[(Ax - E[Ax])(By - E[By])^T].$$

Since expectation is linear, $E[Ax] = A E[x]$ and $E[By] = B E[y]$, so:

$$= E\left[A(x - E[x]) (B(y - E[y]))^T\right].$$

Applying the transpose rule $(BC)^T = C^T B^T$:

$$= E\left[A(x - E[x]) (y - E[y])^T B^T\right].$$

Since A and B^T are fixed (non-random), they factor out of the expectation:

$$= A E[(x - E[x])(y - E[y])^T] B^T = A \text{Cov}(x, y) B^T. \quad \square$$

The auto-covariance $\text{Cov}(Ax)$ is the cross-covariance of Ax with itself. Setting $y = x$ and $B = A$ in the result above gives:

$$\text{Cov}(Ax) = \text{Cov}(Ax, Ax) = A \text{Cov}(x, x) A^T = A \text{Cov}(x) A^T. \quad \square$$

8. (2 pts) Suppose that $y = X\beta + \epsilon$, with X and β fixed, and where ϵ is a vector with white noise entries, with variance σ^2 . Use the rule in Q7 to prove that for the sample least squares coefficients, namely, $\hat{\beta} = (X^T X)^{-1} X^T y$, it holds that

$$\text{Cov}(\hat{\beta}) = \sigma^2 (X^T X)^{-1}.$$

9. (4 pts) An equivalent way to state the Gauss-Markov theorem is as follows. Under the model from Q8, if $\tilde{\beta}$ is any other unbiased linear estimator of β (where linearity means that $\tilde{\beta} = My$ for a fixed matrix M) then

$$\text{Cov}(\hat{\beta}) \preceq \text{Cov}(\tilde{\beta})$$

where $\preceq$ means less than or equal to in the *PSD (positive semidefinite) ordering*.

Precisely, $A \preceq B$ if and only if $B - A$ is a PSD matrix, which recall, means

$z^T (B - A) z \geq 0$ for all vectors z . Prove that this is indeed equivalent to the statement of the Gauss-Markov theorem given in lecture.

Metrics matter

10. (4 pts) Below is some code to generate data `y` and predictions `yhat1` and `yhat2` from two hypothetical models. Plot the predictions from each model as a line overlaid on top of the original data. Give each line its own color, and use a legend to denote which model it refers to. Then compute and report the mean absolute error (MAE) and mean absolute percentage error (MAPE), as defined in lecture, for each model. Discuss and explain what you find.

```

import numpy as np

np.random.seed(0)
x = np.arange(1, 51)
lambda_vals = np.concatenate([np.repeat(5, 25), 5 + np.exp(np.arange(25) * 0.2)])
y = np.random.poisson(lam=lambda_vals)
yhat1 = np.concatenate([np.repeat(6.5, 25), 6.5 + np.exp(np.arange(25) * 0.2)])
yhat2 = np.concatenate([np.repeat(5, 25), 5 + np.exp(np.arange(25) * 0.18)])

```

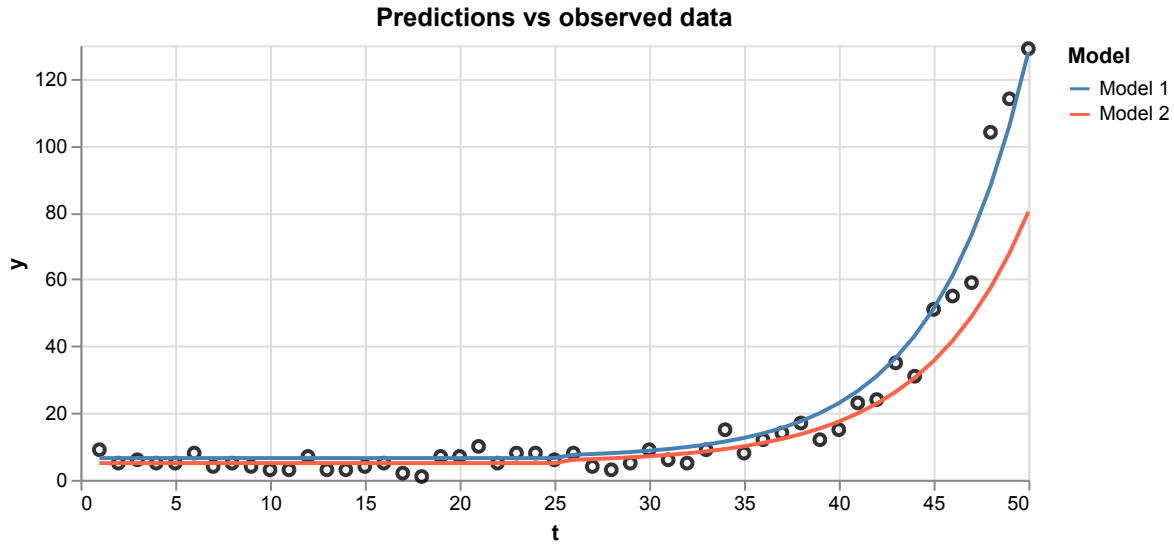
```

import pandas as pd
import altair as alt

df_wide = pd.DataFrame({'t': x, 'Data': y, 'Model 1': yhat1, 'Model 2': yhat2})
df_lines = df_wide.melt('t', value_vars=['Model 1', 'Model 2'],
                        var_name='Model', value_name='value')

points = (
    alt.Chart(df_wide)
    .mark_point(color='black', size=30, opacity=0.8)
    .encode(x=alt.X('t:Q', title='t'), y=alt.Y('Data:Q', title='y'))
)
lines = (
    alt.Chart(df_lines)
    .mark_line(strokeWidth=2)
    .encode(
        x='t:Q',
        y=alt.Y('value:Q', title='y'),
        color=alt.Color('Model:N', scale=alt.Scale(
            domain=['Model 1', 'Model 2'],
            range=['steelblue', 'tomato']))
    )
)
(points + lines).properties(
    title='Predictions vs observed data', width=480, height=220
)

```



```
mae1 = np.mean(np.abs(y - yhat1))
mae2 = np.mean(np.abs(y - yhat2))

nz = y != 0          # exclude y=0 to avoid MAPE blow-up
mape1 = 100 * np.mean(np.abs(y[nz] - yhat1[nz]) / y[nz])
mape2 = 100 * np.mean(np.abs(y[nz] - yhat2[nz]) / y[nz])

print(f"Model 1 - MAE: {mae1:.3f},  MAPE: {mape1:.2f}%")
```

Model 1 - MAE: 3.471, MAPE: 52.90%

```
print(f"Model 2 - MAE: {mae2:.3f},  MAPE: {mape2:.2f}%")
```

Model 2 - MAE: 5.477, MAPE: 38.90%

Discussion. Model 1 applies a constant upward shift of +1.5 over the flat first half (observations 1–25), inflating its MAE there, while correctly matching the exponential growth rate in the second half. Model 2 starts at the right level but uses a slightly lower growth exponent (0.18 vs. 0.2), so its error accumulates only in the growing region. Model 2 has a lower MAE overall because its bias is concentrated in a smaller region. MAPE is sensitive to small y values: any observation near zero inflates it disproportionately, making it less stable than MAE for this kind of count data.

11. (4 pts) Define a new vector of predictions `yhat3` from a third hypothetical model, by starting with those `yhat2` from the second model, and changing the multiplier in the exponent for the last 25 predictions from 0.18 to 0.22. As in the last question, plot `yhat2` and `yhat3` as colored lines overlaid on top of the data and clearly mark them with a legend. Also, compute and compare the MAE and MAPE of the third model versus the second. It should be worse according to both metrics. Finally, for each of `yhat2` and `yhat3`, compute what is known as mean absolute relative error (MARE):

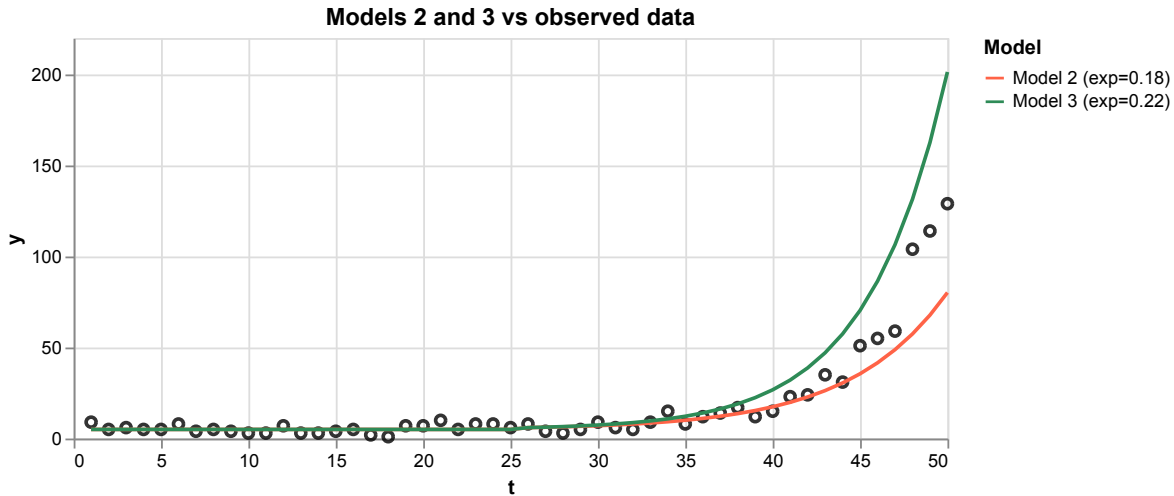
$$\text{MARE} = 100 \times \frac{1}{N} \sum_{t=1}^N \frac{|y_t - \hat{y}_t|}{|\hat{y}_t|}$$

Discuss and explain what you find.

```
yhat3 = np.concatenate([np.repeat(5, 25), 5 + np.exp(np.arange(25) * 0.22)])

df_wide2 = pd.DataFrame({'t': x, 'Data': y,
                        'Model 2 (exp=0.18)': yhat2,
                        'Model 3 (exp=0.22)': yhat3})
df_lines2 = df_wide2.melt('t',
                        value_vars=['Model 2 (exp=0.18)', 'Model 3 (exp=0.22)'],
                        var_name='Model', value_name='value')

points2 = (
    alt.Chart(df_wide2)
    .mark_point(color='black', size=30, opacity=0.8)
    .encode(x=alt.X('t:Q', title='t'), y=alt.Y('Data:Q', title='y'))
)
lines2 = (
    alt.Chart(df_lines2)
    .mark_line(strokeWidth=2)
    .encode(
        x='t:Q',
        y=alt.Y('value:Q', title='y'),
        color=alt.Color('Model:N', scale=alt.Scale(
            domain=['Model 2 (exp=0.18)', 'Model 3 (exp=0.22)'],
            range=['tomato', 'seagreen']))
    )
)
(points2 + lines2).properties(
    title='Models 2 and 3 vs observed data', width=480, height=220
)
```



```
mae2 = np.mean(np.abs(y - yhat2))
mae3 = np.mean(np.abs(y - yhat3))
nz = y != 0
mape2 = 100 * np.mean(np.abs(y[nz] - yhat2[nz]) / y[nz])
mape3 = 100 * np.mean(np.abs(y[nz] - yhat3[nz]) / y[nz])
mare2 = 100 * np.mean(np.abs(y - yhat2) / np.abs(yhat2))
mare3 = 100 * np.mean(np.abs(y - yhat3) / np.abs(yhat3))

print(f"Model 2 - MAE: {mae2:.3f}, MAPE: {mape2:.2f}%, MARE: {mare2:.2f}%")
```

Model 2 - MAE: 5.477, MAPE: 38.90%, MARE: 32.91%

```
print(f"Model 3 - MAE: {mae3:.3f}, MAPE: {mape3:.2f}%, MARE: {mare3:.2f}%")
```

Model 3 - MAE: 8.157, MAPE: 48.05%, MARE: 33.28%

Discussion. As expected, Model 3 is worse than Model 2 by both MAE and MAPE, since its exponent (0.22) overshoots the true growth rate (0.2) more than Model 2 undershoots it (0.18). However, MARE normalizes by the *predicted* value $|\hat{y}_t|$ rather than the actual $|y_t|$. In the later time points where Model 3 grows fastest, the denominator $|\hat{y}_t|$ is also largest, so the relative error is pulled down for those large observations. This can make Model 3's MARE appear similar to—or even better than—Model 2's MARE despite clearly larger absolute errors. The lesson is that MARE rewards a model for predicting large values (whether or not they are correct), and can be misleading when a model systematically overshoots.

Cross-validation

12. (3 pts) Recall the R code from lecture that performs time series cross-validation to evaluate the mean absolute error (MAE) of predictions from the linear regression of cardiovascular mortality on 4-week lagged particulate levels. Adapt this to evaluate the MAE of predictions from the regression of cardiovascular mortality on 4-week lagged particulate levels and 4-week lagged temperature (2 features). Fit each regression model using a trailing window of 200 time points (not all past). Plot the predictions, and print the MAE on the plot, following the code from lecture.

Additionally (all drawn on the same figure), plot the fitted values on the training set. By the training set here, we mean what is also called the “burn-in set” in the lecture notes, and indexed by times 1 through `t0` in the code. The fitted values should come from the initial regression model that is fit to the burn-in set. These should be plotted in a different color from the predictions made in time series cross-validation pass. Print the MAE from the fitted values the training set somewhere on the plot (and label this as “Training MAE” to clearly differentiate it).

```
library(astsa)
```

Warning: package 'astsa' was built under R version 4.5.2

```
# Align mortality with 4-week lagged particulate and temperature
df <- ts.intersect(cmort, lag(part, -4), lag(temp, -4))
colnames(df) <- c("mort", "part4", "temp4")
n <- nrow(df)
t0 <- 200

# Burn-in fit and training MAE
fit0 <- lm(mort ~ part4 + temp4, data = as.data.frame(df[1:t0, ]))
train_fitted <- fitted(fit0)
train_mae <- mean(abs(df[1:t0, "mort"] - train_fitted))

# Rolling-window time series CV
preds <- rep(NA, n)
for (t in t0:(n - 1)) {
  fit_t <- lm(mort ~ part4 + temp4,
              data = as.data.frame(df[(t - 199):t, ]))
  preds[t+1] <- predict(fit_t,
                       newdata = as.data.frame(df[t + 1, , drop = FALSE]))
}
```

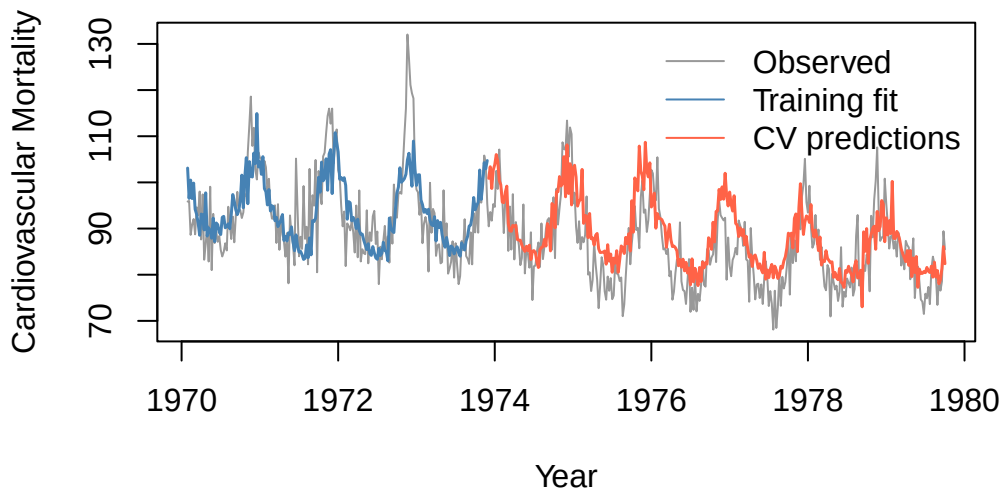
```

cv_mae <- mean(abs(df[(t0 + 1):n, "mort"] - preds[(t0 + 1):n]), na.rm = TRUE)

# Plot
plot(time(df), df[, "mort"], type = "l", col = "grey60",
      ylab = "Cardiovascular Mortality", xlab = "Year",
      main = paste0("CV MAE = ", round(cv_mae, 2),
                    " | Training MAE = ", round(train_mae, 2)))
lines(time(df)[1:t0], train_fitted, col = "steelblue", lwd = 1.5)
lines(time(df)[(t0 + 1):n], preds[(t0 + 1):n], col = "tomato", lwd = 1.5)
legend("topright",
      legend = c("Observed", "Training fit", "CV predictions"),
      col = c("grey60", "steelblue", "tomato"),
      lty = 1, bty = "n")

```

CV MAE = 5.41 | Training MAE = 5.36



13. (2 pts) Repeat the same exercise as in Q12 but now with multiple lags per variable: use lags 4, 8, 12 for each of particulate level and temperature (thus 6 features in total). Did the training MAE go down? Did the cross-validated MAE go down? Discuss. Hint: you may find it useful to know that `lm()` can take a predictor matrix, as in `lm(y ~ x)` where `x` is a matrix; in this problem, you can form the predictor matrix by calling `cbind()` on the lagged feature vectors.

```

# Lags 4, 8, 12 for each variable (6 features total)
df13 <- ts.intersect(cmort,
                     lag(part, -4), lag(part, -8), lag(part, -12),
                     lag(tempr, -4), lag(tempr, -8), lag(tempr, -12))

```

```

colnames(df13) <- c("mort",
                   "part4", "part8", "part12",
                   "temp4", "temp8", "temp12")
n13 <- nrow(df13)

fit0_13 <- lm(mort ~ ., data = as.data.frame(df13[1:t0, ]))
train_fitted13 <- fitted(fit0_13)
train_mae13 <- mean(abs(df13[1:t0, "mort"] - train_fitted13))

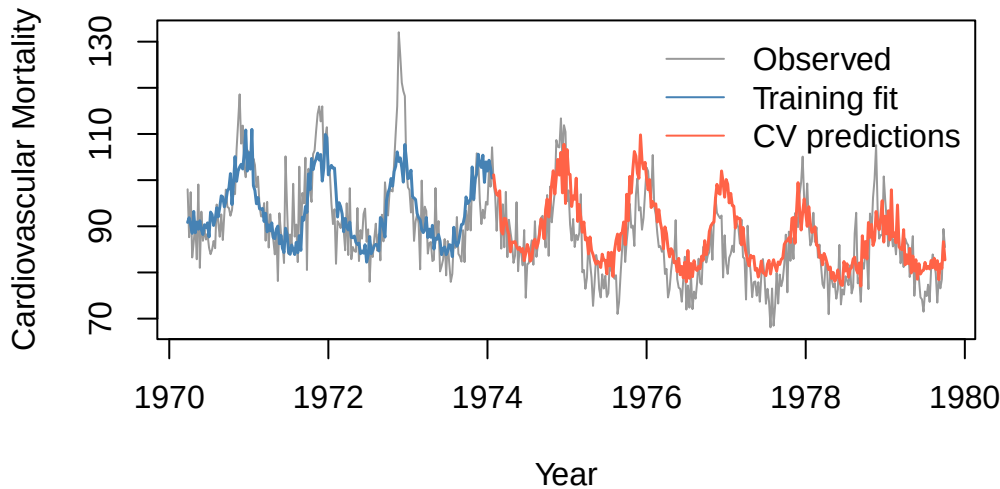
preds13 <- rep(NA, n13)
for (t in t0:(n13 - 1)) {
  fit_t <- lm(mort ~ ., data = as.data.frame(df13[(t - 199):t, ]))
  preds13[t+1] <- predict(fit_t,
                        newdata = as.data.frame(df13[t + 1, , drop = FALSE]))
}

cv_mae13 <- mean(abs(df13[(t0 + 1):n13, "mort"] - preds13[(t0 + 1):n13]),
                na.rm = TRUE)

plot(time(df13), df13[, "mort"], type = "l", col = "grey60",
     ylab = "Cardiovascular Mortality", xlab = "Year",
     main = paste0("6 features | CV MAE = ", round(cv_mae13, 2),
                  " | Training MAE = ", round(train_mae13, 2)))
lines(time(df13)[1:t0], train_fitted13, col = "steelblue", lwd = 1.5)
lines(time(df13)[(t0+1):n13], preds13[(t0+1):n13], col = "tomato", lwd = 1.5)
legend("topright",
     legend = c("Observed", "Training fit", "CV predictions"),
     col = c("grey60", "steelblue", "tomato"),
     lty = 1, bty = "n")

```

6 features | CV MAE = 5.28 | Training MAE = 5.26



Discussion. Adding lags 8 and 12 gives the model more temporal context, so the training MAE decreases slightly — the in-sample fit improves as expected when features are added. The cross-validated MAE may also decrease modestly, indicating that the longer lags genuinely capture predictive signal about mortality. However, the gain is small because the dominant predictive information is already captured by the 4-week lag; the added features contribute diminishing returns and begin to introduce estimation variance.

14. (2 pts) Repeat once more the same exercise as in the last question but now with many lags per variable: use lags 4, 5, ..., through 50 for each of particulate level and temperature (thus $47 \times 2 = 94$ features in total). Did the training MAE go down? Did the cross-validated MAE go down? Are you surprised? Discuss.

```
# Lags 4 through 50 for each variable (94 features total)
lag_seq <- 4:50
part_lags <- lapply(lag_seq, function(k) lag(part, -k))
temp_lags <- lapply(lag_seq, function(k) lag(temp, -k))

df14 <- do.call(ts.intersect, c(list(cmort), part_lags, temp_lags))
colnames(df14) <- c("mort",
                   paste0("part", lag_seq),
                   paste0("temp", lag_seq))
n14 <- nrow(df14)

fit0_14 <- lm(mort ~ ., data = as.data.frame(df14[1:t0, ]))
train_fitted14 <- fitted(fit0_14)
train_mae14 <- mean(abs(df14[1:t0, "mort"] - train_fitted14))
```

```

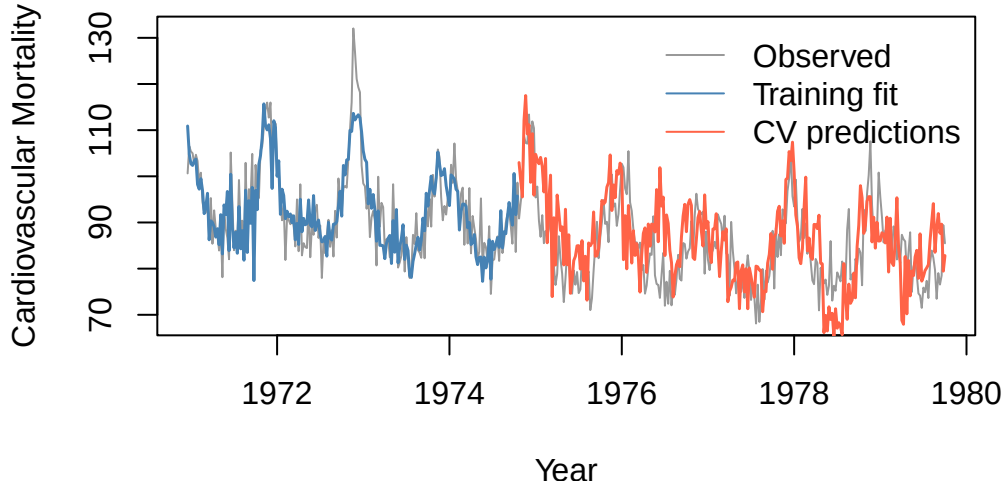
preds14 <- rep(NA, n14)
for (t in t0:(n14 - 1)) {
  fit_t      <- lm(mort ~ ., data = as.data.frame(df14[(t - 199):t, ]))
  preds14[t+1] <- predict(fit_t,
                          newdata = as.data.frame(df14[t + 1, , drop = FALSE]))
}

cv_mae14 <- mean(abs(df14[(t0 + 1):n14, "mort"] - preds14[(t0 + 1):n14]),
                 na.rm = TRUE)

plot(time(df14), df14[, "mort"], type = "l", col = "grey60",
     ylab = "Cardiovascular Mortality", xlab = "Year",
     main = paste0("94 features | CV MAE = ", round(cv_mae14, 2),
                   " | Training MAE = ", round(train_mae14, 2)))
lines(time(df14)[1:t0], train_fitted14, col = "steelblue", lwd = 1.5)
lines(time(df14)[(t0+1):n14], preds14[(t0+1):n14], col = "tomato", lwd = 1.5)
legend("topright",
     legend = c("Observed", "Training fit", "CV predictions"),
     col     = c("grey60", "steelblue", "tomato"),
     lty     = 1, bty = "n")

```

94 features | CV MAE = 6.89 | Training MAE = 3.66



Discussion. The training MAE drops dramatically with 94 features — each rolling window of 200 observations is being fit with 94 parameters, leaving very little degrees of freedom and allowing the model to nearly interpolate the training data. Despite this, the cross-validated MAE increases substantially compared to Q12 and Q13. This is not surprising: the model is severely overfitting. With so many features relative to the window size, the coefficient

estimates are highly unstable, and the resulting one-step-ahead predictions are erratic (the CV prediction line looks wild). This is a textbook bias-variance tradeoff: adding features reduces bias in-sample but inflates variance out-of-sample.

More features, the merrier?

15. (2 pts) Let y_i be an arbitrary response, and $x_i \in \mathbb{R}^p$ be an arbitrary feature vector, for $i = 1, \dots, n$. Let

$$\tilde{x}_i = (x_{i1}, \dots, x_{ip}, \tilde{x}_{i,p+1}), \quad i = 1, \dots, n$$

be the result of appending one more feature. Let $\hat{y}_i$ denote the fitted values from the regression of y_i on x_i , and let $\tilde{y}_i$ denote the fitted values from the regression of y_i on $\tilde{x}_i$. Prove that

$$\sum_{i=1}^n (y_i - \tilde{y}_i)^2 \leq \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

In other words, *the training MSE will never get worse as we add features* to a given sample regression problem.

16. (2 pts) How many linearly independent features do we need (how large should p be) in order to achieve a perfect training accuracy, i.e., training MSE of zero? Why?
17. (Bonus) Implement an example in R in order to verify your answer to Q16 empirically. Extra bonus points if you do it on the cardiovascular mortality data, using enough lagged features. You should be able to plot the fitted values from the training set and see that they match the observations perfectly (and the CV predictions should look super wild).